

Tugas Autoencoder

Training di Kaggle dan Rekonstruksi Citra Fashion-MNIST melalui Decoder

1. Tujuan Tugas

Mahasiswa mampu memahami cara kerja *autoencoder* dalam merekonstruksi citra Fashion-MNIST, melatih model menggunakan Kaggle Notebook, menyimpan model hasil pelatihan, serta menjalankan proses rekonstruksi atau generasi citra melalui *command prompt*/terminal menggunakan bagian *decoder*.

2. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta menggunakan kode praktikum *autoencoder* Fashion-MNIST yang telah diberikan di kelas. Model dilatih di Kaggle Notebook, kemudian model hasil pelatihan disimpan dan digunakan kembali melalui program Python yang dijalankan dari terminal.

Tugas ini tidak hanya menilai kemampuan menjalankan kode, tetapi juga kemampuan mahasiswa memahami hubungan antara *encoder*, *latent space*, *decoder*, dan kualitas hasil rekonstruksi.

3. Alur Umum Tugas

Tugas dikerjakan dengan alur sebagai berikut:

1. Melatih model *autoencoder* di Kaggle Notebook.
2. Menyimpan model hasil training.
3. Mengunduh file model dari Kaggle.
4. Membuat file Python untuk menjalankan *decoder* dari terminal.
5. Menghasilkan citra rekonstruksi atau citra baru dari *latent vector*.
6. Menganalisis hasil rekonstruksi.

4. Instruksi Tugas

4.1. Training Autoencoder di Kaggle

Mahasiswa wajib melakukan training model di Kaggle Notebook.

Langkah yang harus dilakukan:

1. Buat notebook baru di Kaggle.
2. Gunakan dataset Fashion-MNIST.
3. Jalankan kode training *autoencoder*.
4. Latih model minimal selama beberapa epoch sampai *loss* menurun.
5. Tampilkan grafik *training loss*.
6. Tampilkan contoh gambar asli dan hasil rekonstruksi.

4.2. Eksperimen Latent Space

Mahasiswa wajib melakukan training dengan minimal tiga ukuran *latent dimension* berikut:

Latent Dimension
2
8
32

Mahasiswa boleh menambahkan ukuran lain jika diperlukan.

4.3. Menyimpan Model

Setelah training selesai, mahasiswa wajib menyimpan model hasil training.

Contoh perintah penyimpanan model:

```
torch.save(model.state_dict(), "autoencoder_fashion_mnist.pth")
```

Jika encoder dan decoder dibuat sebagai model terpisah, maka bagian decoder juga harus disimpan.

Contoh:

```
torch.save(encoder.state_dict(), "encoder_fashion_mnist.pth")
torch.save(decoder.state_dict(), "decoder_fashion_mnist.pth")
```

File model hasil training harus diunduh dari Kaggle dan digunakan kembali pada komputer lokal atau lingkungan Python lain.

5. Rekonstruksi Melalui Command Prompt atau Terminal

Mahasiswa wajib membuat program Python terpisah, misalnya:

`reconstruct.py`

Program tersebut dijalankan melalui terminal atau command prompt, bukan hanya dari notebook.

5.1. Contoh Perintah Menjalankan Program

Contoh menjalankan program dari terminal:

```
python reconstruct.py --model autoencoder_fashion_mnist.pth --  
index 10
```

Atau jika menggunakan decoder saja:

```
python generate_from_decoder.py --decoder decoder_fashion_mnist.  
pth --z1 0.5 --z2 -1.2
```

5.2. Ketentuan Program Terminal

Program Python yang dibuat mahasiswa minimal harus mampu:

1. membaca file model hasil training;
2. memuat arsitektur model yang sama dengan saat training;
3. memilih satu data Fashion-MNIST berdasarkan indeks;
4. menghasilkan gambar rekonstruksi;
5. menyimpan hasil rekonstruksi sebagai file gambar, misalnya `output.png`;
6. dijalankan melalui terminal atau command prompt.

6. Pilihan Implementasi

Mahasiswa dapat memilih salah satu dari dua bentuk implementasi berikut.

6.1. Opsi A: Rekonstruksi Menggunakan Autoencoder Penuh

Pada opsi ini, mahasiswa menggunakan model *autoencoder* lengkap.

Alur:

input image → *encoder* → *latent vector* → *decoder* → *reconstructed image*

Contoh perintah:

```
python reconstruct.py --model autoencoder_fashion_mnist.pth --  
index 25
```

Program harus menyimpan hasil sebagai:

```
original.png  
reconstructed.png  
comparison.png
```

6.2. Opsi B: Generasi Citra Menggunakan Decoder

Pada opsi ini, mahasiswa hanya menggunakan bagian *decoder*. Mahasiswa memberi nilai *latent vector* melalui terminal, lalu decoder menghasilkan gambar.

Contoh untuk *latent dimension* 2:

```
python generate_from_decoder.py --decoder decoder_fashion_mnist.  
pth --z1 0.5 --z2 -1.2
```

Program harus menghasilkan file:

```
generated_image.png
```

Jika *latent dimension* lebih dari 2, mahasiswa dapat menggunakan argumen berupa daftar angka.

Contoh:

```
python generate_from_decoder.py --decoder decoder_fashion_mnist.  
pth --latent "0.5,-1.2,0.3,0.8"
```

7. Output yang Wajib Dikumpulkan

Mahasiswa wajib mengumpulkan:

1. Notebook Kaggle dalam format `.ipynb`;
2. file model hasil training dengan format `.pth`;
3. file Python untuk terminal, misalnya `reconstruct.py` atau `generate_from_decoder.py`;
4. screenshot proses training di Kaggle;

5. screenshot menjalankan program dari terminal;
6. gambar hasil rekonstruksi atau generasi;
7. laporan PDF maksimal 5 halaman.

8. Isi Laporan

Laporan disusun dengan struktur berikut:

1. Judul tugas;
2. nama dan NIM;
3. penjelasan singkat arsitektur *autoencoder*;
4. penjelasan proses training di Kaggle;
5. hasil grafik *loss*;
6. hasil rekonstruksi dari beberapa ukuran *latent dimension*;
7. penjelasan cara menjalankan program dari terminal;
8. screenshot terminal;
9. analisis hasil;
10. kesimpulan.

9. Panduan Analisis

Mahasiswa tidak perlu menjawab pertanyaan analisis dalam bentuk daftar jawaban singkat. Jawaban ditulis dalam bentuk paragraf naratif pada bagian analisis laporan. Paragraf analisis harus menjelaskan hubungan antara arsitektur *autoencoder*, ukuran *latent space*, proses rekonstruksi, dan hasil eksperimen yang diperoleh.

Analisis minimal mencakup beberapa aspek berikut:

1. peran *encoder* dalam mengubah citra input menjadi representasi laten;
2. peran *decoder* dalam membangun kembali citra dari *latent vector*;
3. makna *latent space* sebagai ruang representasi fitur;
4. alasan mengapa informasi citra perlu disimpan dalam bentuk *latent vector*;
5. pengaruh *latent dimension* yang terlalu kecil terhadap hilangnya detail citra;
6. pengaruh *latent dimension* yang lebih besar terhadap kualitas rekonstruksi;

7. perbandingan hasil rekonstruksi dari beberapa ukuran *latent dimension*;
8. hubungan antara nilai *loss* dan kualitas visual hasil rekonstruksi;
9. perbedaan rekonstruksi yang dijalankan di notebook dan melalui terminal;
10. pentingnya kesesuaian arsitektur model saat training dan inferensi;
11. alasan mengapa *decoder* membutuhkan *latent vector* sebagai input;
12. kualitas citra yang dihasilkan dari *latent vector* acak;
13. refleksi apakah *autoencoder* benar-benar memahami struktur gambar atau hanya belajar menyalin input.

Mahasiswa harus menggunakan bukti dari hasil eksperimen, seperti grafik *loss*, gambar hasil rekonstruksi, hasil generasi dari *decoder*, dan tabel perbandingan *latent dimension*. Analisis tidak boleh hanya berisi definisi teori, tetapi harus dikaitkan dengan hasil yang diperoleh sendiri.

10. Tabel Perbandingan Hasil

Mahasiswa wajib membuat tabel perbandingan hasil eksperimen seperti berikut:

Latent Dimension	Loss Akhir	Kualitas Visual	Catatan
2			
8			
32			

11. Ketentuan Teknis

Ketentuan teknis tugas adalah sebagai berikut:

1. Training dilakukan di Kaggle Notebook.
2. Inferensi atau rekonstruksi dilakukan melalui terminal atau command prompt.
3. File model harus dapat dimuat ulang tanpa melakukan training ulang.
4. Program terminal harus menghasilkan file gambar.
5. Mahasiswa wajib mencantumkan perintah terminal yang digunakan.
6. Mahasiswa wajib menyertakan screenshot terminal sebagai bukti.
7. Mahasiswa tidak boleh hanya menampilkan hasil di notebook.

12. Ketentuan Penulisan Analisis

Analisis harus ditulis dalam bentuk paragraf, bukan dalam bentuk jawaban bernomor. Mahasiswa boleh menggunakan subjudul, tetapi isi analisis harus berupa uraian naratif yang saling terhubung.

Contoh penulisan yang tidak disarankan:

1. Encoder berfungsi untuk mengubah gambar menjadi latent vector.
2. Decoder berfungsi untuk mengubah latent vector menjadi gambar.

Contoh penulisan yang disarankan:

Pada model autoencoder, encoder berperan sebagai bagian yang mereduksi citra Fashion-MNIST ke dalam representasi laten berdimensi lebih kecil. Representasi ini menyimpan informasi penting dari bentuk pakaian, seperti pola umum, kontur, dan struktur visual. Selanjutnya, decoder menggunakan representasi tersebut untuk membangun kembali citra. Berdasarkan hasil eksperimen, ukuran latent dimension yang terlalu kecil menyebabkan sebagian detail citra hilang, sehingga hasil rekonstruksi tampak lebih buram atau kurang jelas.

13. Contoh Struktur Folder Pengumpulan

Struktur folder yang disarankan:

```
nama_nim_autoencoder/  
|-- autoencoder-fashion-mnist.ipynb  
|-- autoencoder_fashion_mnist.pth  
|-- reconstruct.py  
|-- generate_from_decoder.py  
|-- original.png  
|-- reconstructed.png  
|-- generated_image.png  
|-- laporan.pdf  
|-- README.md
```

14. Isi README

File README.md minimal berisi:

1. nama dan NIM;
2. versi Python dan library yang digunakan;

3. cara menjalankan training di Kaggle;
4. cara menjalankan rekonstruksi dari terminal;
5. contoh perintah terminal;
6. daftar file output yang dihasilkan.

15. Rubrik Penilaian Analisis

Bagian analisis dinilai berdasarkan kedalaman pemahaman, keterkaitan dengan hasil eksperimen, dan kemampuan mahasiswa menarik kesimpulan secara kritis.

Aspek	Kriteria Penilaian	Bobot
Pemahaman konsep <i>encoder</i> , <i>decoder</i> , dan <i>latent space</i>	Mahasiswa mampu menjelaskan fungsi masing-masing komponen dengan bahasa sendiri dan mengaitkannya dengan proses rekonstruksi citra.	20%
Analisis pengaruh <i>latent dimension</i>	Mahasiswa mampu menjelaskan dampak <i>latent dimension</i> kecil dan besar terhadap detail citra, kualitas rekonstruksi, serta informasi yang tersimpan.	25%
Keterkaitan antara <i>loss</i> dan kualitas visual	Mahasiswa mampu membandingkan nilai <i>loss</i> dengan hasil visual, serta menjelaskan bahwa nilai <i>loss</i> rendah tidak selalu menjamin hasil visual terbaik.	15%
Analisis inferensi melalui terminal	Mahasiswa mampu menjelaskan perbedaan menjalankan rekonstruksi di notebook dan terminal, termasuk pentingnya menyimpan model, memuat ulang model, dan menjaga kesesuaian arsitektur.	15%
Analisis penggunaan <i>decoder</i>	Mahasiswa mampu menjelaskan mengapa <i>decoder</i> membutuhkan <i>latent vector</i> , serta menganalisis hasil citra dari <i>latent vector</i> acak.	15%
Refleksi kritis	Mahasiswa mampu memberikan pendapat apakah <i>autoencoder</i> memahami struktur gambar atau hanya menyalin input, disertai bukti dari eksperimen.	10%
Total		100%

16. Catatan Penting

Tugas ini tidak dinilai hanya dari keberhasilan menjalankan model, tetapi juga dari kemampuan mahasiswa menjelaskan proses yang terjadi di dalam *autoencoder*.

Mahasiswa harus memahami bahwa proses rekonstruksi tidak hanya terjadi karena model menampilkan gambar, tetapi karena data melalui tahapan:

$$\textit{input image} \rightarrow \textit{encoder} \rightarrow \textit{latent space} \rightarrow \textit{decoder} \rightarrow \textit{reconstructed image}$$

Ketika mahasiswa menjalankan *decoder* melalui terminal, mahasiswa diharapkan memahami bahwa *decoder* dapat menghasilkan citra dari representasi laten yang telah dipelajari selama proses training.